圖走訪與連通性

從 BFS 的距離層、DFS 的時間區間，走向拓樸排序、強連通分量與單點或單邊故障分析。

本章依序研究 **BFS → DFS → 拓樸排序 → SCC → low、割點、橋與雙連通區塊**。

前置知識為佇列、堆疊、遞迴、集合與歸納證明。每次走訪都要先固定圖模型與鄰接順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先問輸出，再選擇走訪資訊

問題	保留的資訊	證明核心
最少幾條邊到達？	BFS 距離與 parent	FIFO 層次不變量
何時進入與完成？	DFS 顏色、森林與時間	括號與 white-path 定理
哪些工作可先做？	DAG 的完成逆序或剩餘入度	每條邊遵守順序
哪些頂點互相可達？	SCC 與縮點圖	完成時間與分量邊界
移除頂點或邊會斷開嗎？	無向 low 與邊堆疊	子樹能否回到祖先

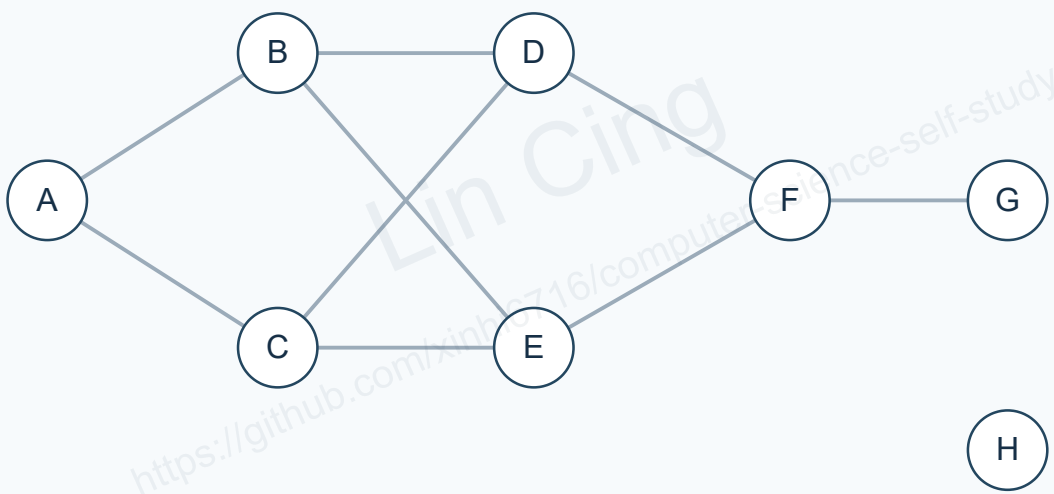
BFS 的 d 是距離；DFS 的 d 是發現時間。SCC lowlink 與無向 low 的更新規則必須分開。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

圖的模型先決定邊如何計數

本節固定無權、無向、無自環且無平行邊的圖。$V = \{A, B, C, D, E, F, G, H\}$，$|V| = 8$，$|E| = 9$。邊集合為 $\{AB, AC, BD, BE, CD, CE, DF, EF, FG\}$。

固定圖 G，起點 A，H 孤立



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8 個頂點，9 條無向邊。鄰居按字母順序。

問題是從 A 出發，求到各點的最少邊數與一條對應路徑。H 沒有任何邊。

鄰接串列與矩陣記錄相同關係

串列 `Adj[u]` 列出 u 的鄰居。矩陣 $M[u, v] = 1$ 當且僅當有邊 uv。本例矩陣對稱、對角線為 0。

同一張圖的兩種表示

鄰接串列

A: B, C

B: A, D, E

C: A, D, E

D: B, C, F

E: B, C, F

F: D, E, G

G: F

H: ∅

	A	B	C	D	E	F	G	H
A	0	1	1	0	0	0	0	0
B	1	0	0	1	1	0	0	0
C	1	0	0	1	1	0	0	0
D	0	1	1	0	0	1	0	0
E	0	1	1	0	0	1	0	0
F	0	0	0	1	1	0	1	0
G	0	0	0	0	0	1	0	0
H	0	0	0	0	0	0	0	0

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

串列共 18 個鄰接項目。矩陣 8 × 8 = 64 格，且 A = Aᵀ。

無向每條邊存兩個鄰接項目，所以總數為 $2|E| = 18$。有向圖只將弧 $u \to v$ 存在 `Adj[u]`，總數為 $|E|$。

查邊與列舉鄰居需要不同成本

令 $n = |V|$，$m = |E|$，使用普通未排序串列與直接索引矩陣。

操作	鄰接串列	鄰接矩陣
儲存	$\Theta(n + m)$	$\Theta(n^2)$
查詢 $u \to v$	最壞 $O(1 + \deg^+(u))$	$\Theta(1)$
列出 u 的所有鄰居	$\Theta(1 + \deg^+(u))$	$\Theta(n)$

無向圖以 $\deg(u)$ 代替出度。常數 1 包含空串列的處理。有向矩陣不必對稱。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

自環和平行邊必須說清楚表示規則

若改採多重圖，每條有向平行弧各占一個串列項目，自環 $u \to u$ 也占一項。無向圖採每條邊兩個端點 incidence 的慣例，自環在同一串列存兩項，所以總數仍是 $2m$。

二元矩陣只保存「是否相鄰」，會丟失平行邊數。可改存重數，但邊身分或各別權重還要額外結構。若無向自環改只存一次，總項目數是 $2m - \ell$，其中 ℓ 是自環數。

BFS 只在白點首次發現時入隊。自環和平行邊不使頂點重複入隊，卻會增加串列掃描次數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

顏色、距離與 parent 各有一個角色

白色表示未發現，灰色表示已發現但尚未掃描完鄰居，黑色表示鄰居全部掃描完。只有 `WHITE → GRAY → BLACK`。

$d[v]$ 是從 A 到 v 的邊數估計，初值為 ∞。$\pi[v]$ 記首次發現 v 的點，初值為 NIL。起點設 `d[A]=0`、灰色、入隊。

在每次 while 條件測試時，Q 恰包含所有灰點。剛出隊但仍在掃描的 u 也是灰色，因此不能把這項等式宣稱為每個程式瞬間都成立。筆記 PDF p34 的 DFS d 表示發現時間，本節 BFS d 表示距離。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先標灰，再入隊，才能只發現一次

```
for v in V: color[v]=WHITE; d[v]=∞; π[v]=NIL
color[A]=GRAY; d[A]=0; Q=[A]
while Q is not empty:
    u=dequeue_front(Q)
    for v in Adj[u] in alphabetical order:
        if color[v]==WHITE:
            color[v]=GRAY; d[v]=d[u]+1; π[v]=u
            enqueue_back(Q,v)
    color[u]=BLACK
```

佇列需用 O(1) 的頭尾操作。用陣列每次刪第一格並搬移其他元素，會失去這項成本假設。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

前兩次掃描確定距離 1 與 2

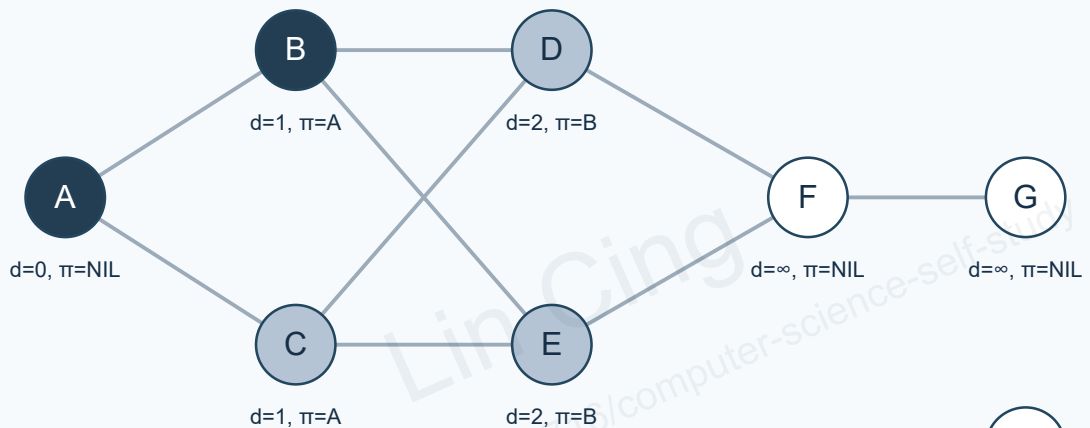
初始 `Q=[A]`。掃描 A 的 B、C，兩點由白轉灰，距離均為 1、parent 均為 A。完成後 A 變黑，`Q=[B,C]`。

接著出隊 B。A 已黑，D、E 仍白，所以令 `d[D]=d[E]=2`、`π[D]=π[E]=B`。完成後 `Q=[C,D,E]`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

前兩次掃描確定距離 1 與 2 （續）

掃描 B 完成後，Q = [C, D, E]



黑色 A,B 已完成。灰色 C,D,E 已發現。白色 F,G,H 未發現。

C 雖已在隊內，仍要先於 D、E 出隊。

完整軌跡記錄每次掃描的結果

完整 FIFO 佇列軌跡

完成掃描	本次新發現	Q 隊首 → 隊尾	已完成黑點
init	A	[A]	∅
A	B,C	[B, C]	A
B	D,E	[C, D, E]	A,B
C	∅	[D, E]	A,B,C
D	F	[E, F]	A,B,C,D
E	∅	[F]	A,B,C,D,E
F	G	[G]	A,B,C,D,E,F
G	∅	[]	A,B,C,D,E,F,G

每列均在 while 測試前，Q 恰好包含灰點。H 始終白色。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

佇列軌跡的逐步解讀

掃描 C 時，D、E 已灰，不更改距離或 parent。掃描 D 首次發現 F，令 `d[F]=3`，`π[F]=D`。掃描 E 時 F 已灰。掃描 F 發現 G，令 `d[G]=4`，`π[G]=F`。

G 掃描完後 Q 為空。七個可達點各入隊一次，H 從未入隊。

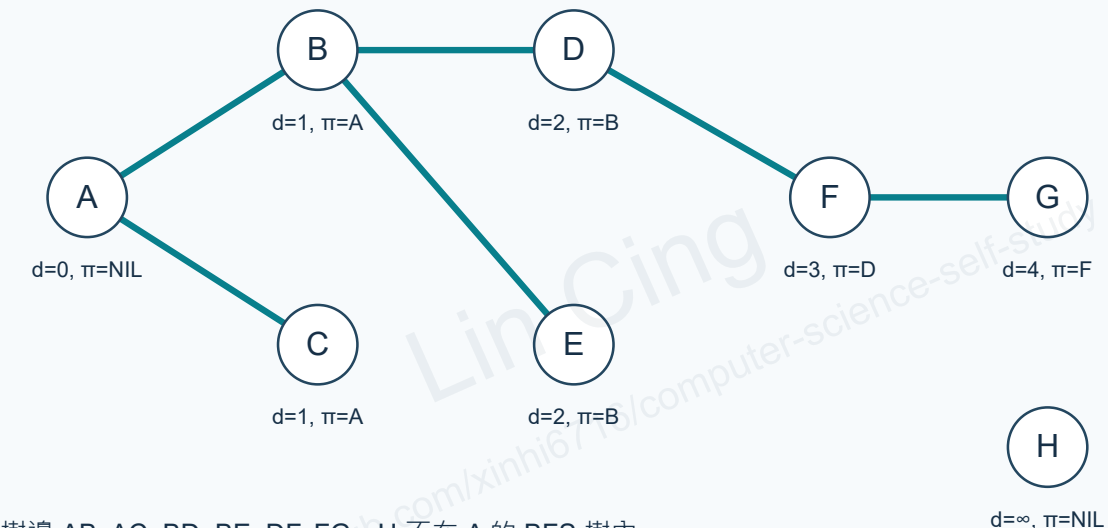
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

parent 樹保留一條最短路徑

v	A	B	C	D	E	F	G	H
d[v]	0	1	1	2	2	3	4	∞
π[v]	NIL	A	A	B	B	D	F	NIL

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

parent 樹保留一條最短路徑 （續）



從 G 沿 parent 回溯得到 `G,F,D,B,A`，反轉後為 `A,B,D,F,G`，共 4 條邊。`π=NIL` 也可能代表起點，須先檢查目標是否就是 A，再判斷不可達。

FIFO 保持非遞減的距離順序

若 Q 非空，隊內距離非遞減，且隊尾距離至多比隊首多 1。Q 只會同時含相鄰兩層。

理由是出隊刪除最前面的最小值。掃描距離 k 的 u 時，新點一律標成 k+1，附在隊尾。當層尚未處理的點留在前方，下一層的新點留在後方。

例如掃描 B 後 Q 的距離是 `[1,2,2]`。掃描 D 後是 `[2,3]`。每個距離 k 的點，都會先於距離 k+1 的點被掃描。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最短邊數需要上下兩個界

令 $\delta(A, v)$ 為圖中最少邊數。對已發現點，parent 鏈提供一條長度 $d[v]$ 的實際路徑，所以 $\delta(A, v) \leq d[v]$。

再按真實距離 k 歸納。k=0 的 A 正確。若 $\delta(A, v) = k > 0$，最短路徑上前一點 u 的距離為 k−1。由歸納假設 u 正確，且會出隊掃描。

掃描 u 時，若 v 白，就設 $d[v] = k$。若 v 已被發現，FIFO 保證先前掃描的點距離至多 k−1，所以 $d[v] \leq k$。結合第一個界，$d[v] = k$。因此所有可達點都會被發現，且得到最短邊數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

鄰居順序改變樹，距離不變

把各點鄰居改成反字母順序。A 先發現 C 再發現 B。C 先發現 E、D，因此 $\pi[D]=\pi[E]=C$。E 先於 D 掃描，所以 $\pi[F]=E$。

出隊順序變為 A,C,B,E,D,F,G。到 G 的樹路徑變為 A,C,E,F,G。距離向量仍是 $(0,1,1,2,2,3,4,\infty)$。

兩種樹都通過最短路徑證明。δ 是原圖與起點決定的數值，parent 是演算法在同長路徑間的選擇。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一次 BFS 不會自動走完所有分量

從 A 結束後，H 仍白、$d[H] = \infty$，因為沒有 A 到 H 的路徑。

若要列出無向圖的連通分量，先全體初始化一次。再依序檢查頂點，每遇白點就以它為根執行 BFS，寫入新的分量編號。本例分量是 {A,B,C,D,E,F,G} 與 {H}。

每棵樹內的距離應解讀為到各自根的距離。對有向圖，這個外層迴圈只產生依順序而變的可達搜尋森林，不能據此宣稱強連通分量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

表示方式決定 BFS 掃描成本

設 V_R 為從起點可達的點。串列版本初始化 $\Theta(n)$，每個可達點只出隊一次，每個對應鄰接項目只掃描一次。總時間 $\Theta(n + \sum_{u \in V_R} |Adj[u]|)$，上界 $O(n + m)$。

矩陣版本每個出隊點掃一整列，時間 $\Theta(n + n|V_R|)$，最壞 $\Theta(n^2)$。兩者額外的顏色、距離、parent 與 queue 空間皆為 $O(n)$，不含圖儲存。

本例串列掃 18 個項目，矩陣掃 $7 \times 8 = 56$ 格，兩者都初始化 8 點。若連 H 一起完成全森林，矩陣掃 64 格，串列仍掃 18 項。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

BFS 最佳化的是邊數

若每條邊權重皆為同一個正數 c，最少邊數路徑也最小化總權重，路徑成本為 $c\,d[v]$。不同權重時不能套用此結論。

例如有向邊 s→t 權重 10、s→x 權重 1、x→t 權重 1。BFS 選 1 條邊的直接路徑，但最小總權重路徑有 2 條邊、成本 2。

本節固定圖與五張 SVG 是原創例子。CLRS §22.1–22.2 提供表示、演算法與證明依據。MIT04 L14 與 MIT05 L17 p43–46、59 以單位權重與 FIFO 說明 BFS。筆記 p40 在 Edmonds–Karp 中使用 BFS 選最少邊擴充路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DFS 的輸出是森林與時間區間

給定圖 G=(V,E)，完整 DFS 對每個頂點恰好發現一次、完成一次。輸出 parent、發現時間 d、完成時間 f。樹邊給出可達路徑，時間區間給出祖先關係。DFS 樹路徑通常不是最短路徑。

本節先完整追蹤一張有環有向圖，再刪邊得到 DAG，用同一組時間驗證拓樸排序。

來源 · note-original; CLRS3-22 §§22.3–22.4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先固定圖模型與走訪順序

本例為有限有向圖，沒有平行邊或自迴路。完整 DFS 的外層順序 A,B,C,D,E,F。鄰接表順序固定如下，括號內的先後不可互換。

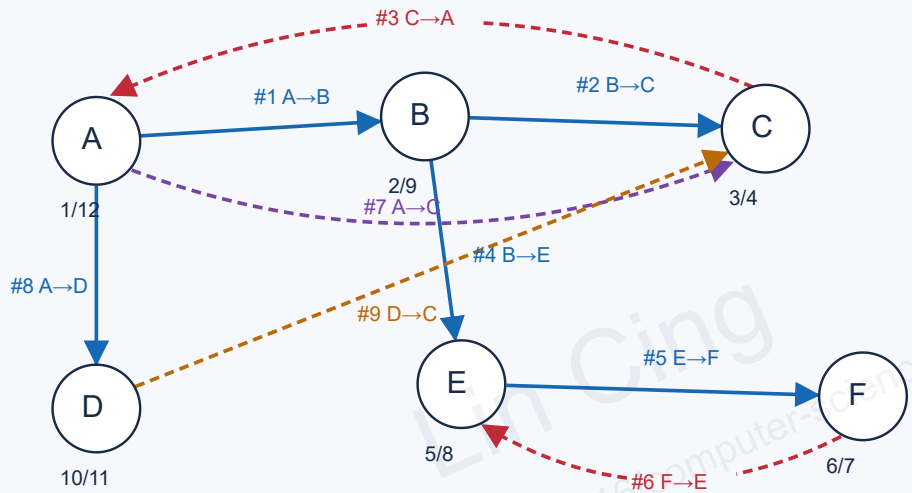
```
A:[B,C,D] B:[C,E] C:[A] D:[C] E:[F] F:[E]
```

箭頭表示一條原圖邊。樹邊以實線藍色、back 以紅色、forward 以紫色、cross 以橙色顯示。分類屬於這一次 DFS，換順序可改變分類。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先固定圖模型與走訪順序 （續）

固定順序、掃邊編號與 d/f



tree #1,#2,#4,#5,#8 back #3,#6 forward #7 cross #9

A:[B,C,D] B:[C,E] C:[A] D:[C] E:[F] F:[E]

來源 • MIT2004 pp.2–6; MIT2005 pp.2–5; CLRS3-22。

三色表示遞迴生命週期

WHITE 尚未被發現。GRAY 已發現但呼叫尚未返回。BLACK 已掃完所有出邊。探索邊時的來源一定是目前最深的 GRAY 頂點。所有 GRAY 頂點恰好形成呼叫堆疊的一條祖先鏈。

時間只在發現、完成時加一，不在每次掃邊時加一。每點滿足 $1 \le d[u] < f[u] \le 2|V|$，全圖共 $2|V|$ 個互異時間。

來源 · note-original; CLRS3-22 § §22.3–22.4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整 DFS 與 VISIT 的責任

```
DFS(G):
  all colors = WHITE; all parents = NIL; time = 0
  for u in fixed vertex order:
    if color[u] == WHITE: VISIT(u)
VISIT(u):
  time += 1; d[u] = time; color[u] = GRAY
  for v in Adj[u] in fixed order:
    if color[v] == WHITE:
      parent[v] = u; VISIT(v)
  color[u] = BLACK; time += 1; f[u] = time
```

VISIT(u) 只探索從當下白色 u 可延伸的白色頂點。外層迴圈補齊未訪問頂點，因此有向圖可能得到多棵樹。不同 DFS 樹不代表強連通分量；跨樹仍可能有指向先前 BLACK 頂點的邊。

來源 • note-original; CLRS3-22 §§22.3–22.4。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

逐輪堆疊就是時間線

t	event	stack after
1	discover A	A
2	discover B	AB
3	discover C	ABC
4	finish C	AB
5	discover E	ABE
6	discover F	ABEF
7	finish F	ABE
8	finish E	AB
9	finish B	A
10	discover D	AD

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逐輪堆疊就是時間線 （續）

t	event	stack after
11	finish D	A
12	finish A	∅

每次發現先 push，每次完成再 pop。t=4 返回 B 後，B 的鄰接掃描器才繼續處理 E。t=9 返回 A 後，A 還要掃 C、D。完成不等於整個演算法結束。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逐輪堆疊就是時間線 （續）

完整十二事件與堆疊

每格顯示事件發生後的堆疊，左為底、右為頂。

t= 1 discover A	A				GRAY = A
t= 2 discover B	A	B			GRAY = AB
t= 3 discover C	A	B	C		GRAY = ABC
t= 4 finish C	A	B			GRAY = AB
t= 5 discover E	A	B	E		GRAY = ABE
t= 6 discover F	A	B	E	F	GRAY = ABEF
t= 7 finish F	A	B	E		GRAY = ABE
t= 8 finish E	A	B			GRAY = AB
t= 9 finish B	A				GRAY = A
t=10 discover D	A	D			GRAY = AD
t=11 finish D	A				GRAY = A
t=12 finish A	∅				GRAY = ∅

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

每條邊都要掃，遇到已訪問也不跳表

#	edge	target	type	stack	clock
1	AB	W	tree	A	1
2	BC	W	tree	AB	2
3	CA	G	back	ABC	3
4	BE	W	tree	AB	4
5	EF	W	tree	ABE	5

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

掃描餘下邊並比較分類

#	edge	target	type	stack	clock
6	FE	G	back	ABEF	6
7	AC	B	forward	A	9
8	AD	W	tree	A	9
9	DC	B	cross	AD	10

clock 是掃邊當刻已發生的最後一個時間事件。target 的 W/G/B 是掃邊當刻的顏色。A→C 在 C 完成後才掃到，因此是 forward；D→C 也到 BLACK，但 C 不是 D 的後代，因此是 cross。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

最後時間與森林對齊

vertex	d	f	parent
A	1	12	NIL
B	2	9	A
C	3	4	B
D	10	11	A
E	5	8	B
F	6	7	E

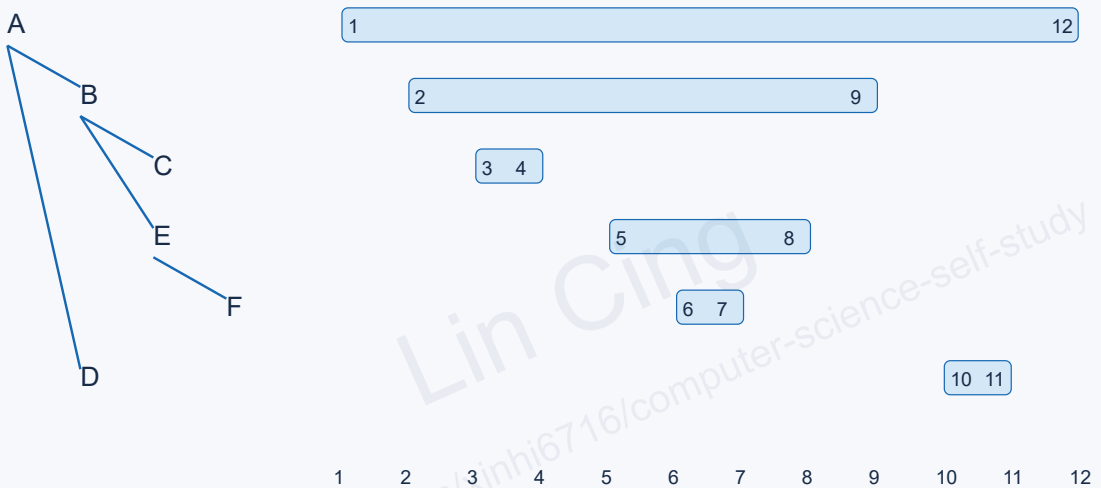
樹邊為 A→B、B→C、B→E、E→F、A→D。發現順序 A B C E F D；完成順序 C F E B D A。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最後時間與森林對齊 （續）

呼叫樹對齊 d/f 區間

樹邊與區間共用頂點列；橫軸為事件時間。



(A (B (C C) (E (F F) E) B) (D D) A)

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

括號定理排除交錯區間

對相異頂點 u,v，區間 [d[u],f[u]] 與 [d[v],f[v]] 只能互斥或嚴格包含。包含當且僅當內層頂點是外層頂點的真後代。

證明。假設 d[u]<d[v]。若 f[u]<d[v]，u 已返回，兩區間互斥。否則 v 在 u 尚為 GRAY 時發現，v 的呼叫嵌套於 u 內。遞迴必須先完成 v 才能返回並完成 u，因此 d[u]<d[v]<f[v]<f[u]。交換 u,v 處理另一種發現順序。

本例括號為 `(A (B (C C) (E (F F) E) B) (D D) A)`。`d[u]<d[v]<f[u]<f[v]` 不可能。

來源 · CLRS3-22 § §22.3–22.4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

white-path 定理的時間前提

在 u 剛被發現的瞬間，v 成為 u 的後代，當且僅當存在 u→⋯→v 的路徑，除已轉 GRAY 的起點 u 外，其餘頂點此刻全是 WHITE。若取染色前瞬間，整條路徑均為 WHITE。此處包含 u 自己作零長路徑。

必要性。DFS 樹的 u 到 v 路徑上每個真後代都比 u 晚發現，故此刻為白。充分性。若白路徑存在，取路徑上第一個不是 u 後代的 x，其前驅 w 是 u 或 u 後代。掃 w→x 時，x 若白就立刻遞迴；若非白，它必在 u 發現後、w 完成前被發現。兩者都讓 x 的發現落在 u 活躍期內，矛盾。

只有「原圖有路徑」不夠。C→A→D 可達，但在 d[C]=3 時 A 是 GRAY，D 不會成為 C 後代。

來源 • CLRS3-22 § §22.3–22.4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有向圖的四種邊

類型	掃邊時判別	完成後的關係
tree	目標 WHITE	parent[v]=u
back	目標 GRAY	v 是 u 的祖先，含自迴路
forward	目標 BLACK 且 d[u]<d[v]	真後代但非樹邊
cross	目標 BLACK 且 d[v]<d[u]	無祖先關係

對相異頂點，tree/forward 均滿足 d[u]<d[v]<f[v]<f[u]，因此只看時間不能分辨它們，還需要 parent。back 滿足 d[v]<d[u]<f[u]<f[v]。cross 滿足 d[v]<f[v]<d[u]<f[u]。

來源 • note-original; CLRS3-22 § §22.3–22.4。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

無向邊只有 tree 或 back

無向簡單圖中，一條實體邊在兩端鄰接表各出現一次，必須按第一次遇到的方向分類。已用來發現子節點的那條 parent 邊，反向掃描時不是另一條 back edge。

證明。令 d[u]<d[v] 且 {u,v} 為邊。u 完成前必掃到 v，因此 v 是 u 後代。若第一次從 u 掃向 v，v 尚白，為 tree；若第一次由 v 掃向 u，u 尚灰，為 back。不會有跨子樹邊。

無向環檢測忽略同一條 parent 邊的反向項。多重圖要用 edge ID，平行的另一條邊仍可形成長度 2 的環。自迴路形成長度 1 的環；定理的環定義須與圖模型一致。

來源 • note-original; CLRS3-22 §§22.3–22.4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有向圖有環當且僅當有 back edge

本命題使用有限有向圖，允許自迴路，環是沿箭頭可走回起點且內部點互異的非空路徑。必須對全圖跑完整 DFS。

若存在 u→v back edge，樹中 v 到 u 的有向路徑加上 u→v 成環，自迴路本身就是環。反方向取某個環最早發現的頂點 v，以及環上指向 v 的前驅 u。發現 v 時其餘環頂點皆白，沿環存在白路徑 v→⋯→u。white-path 定理使 u 成為 v 後代，所以 u→v 是 back edge。

本例 C→A 證明 A→B→C→A 成環，F→E 證明 E→F→E 成環。只看到 BLACK 不能推論成環。

來源 • note-original; CLRS3-22 §§22.3–22.4。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

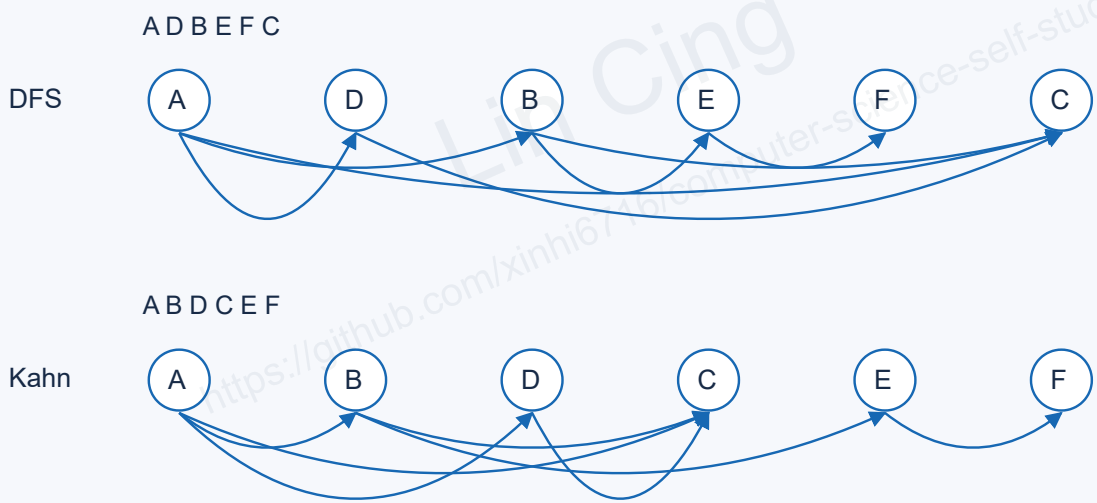
刪除兩條 back edge 得到 DAG

令 G′=G−{C→A,F→E}。鄰接順序不變，所有 d/f 與樹邊也不變。完成逆序為 A D B E F C。逐條邊檢查位置均由左向右，所以這是 G′ 的拓樸序。

原圖相同逆序不是拓樸序，C→A 與 F→E 都逆向。任何圖都能產生完成逆序，只有 DAG 保證其為拓樸序。

同一 DAG 的兩種拓樸序

G′ = G − {C→A, F→E}



每條弧線由左指向右；順序可以不唯一。

為何完成逆序可作拓樸排序

DAG 的每條邊 u→v 在掃描時，v 不可能是 GRAY，否則產生 back edge 與有向環。若 v 是 WHITE，遞迴先完成 v 再完成 u。若 v 是 BLACK，v 已完成而 u 尚未完成。因此每條邊均有 f[u]>f[v]。按 f 遞減排列便保證 u 排在 v 前。

實作在完成時 append，再整體 reverse，或完成時插到鏈結串列前端。不要用陣列反覆插入最前方，否則可能多花 $\Theta(V^2)$。若碰到 back edge，回報環並拒絕輸出拓樸序。

來源 • CLRS3-22 § §22.3–22.4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kahn 用剩餘入度看同一個 DAG

Kahn 重複取剩餘入度 0 的點，刪除它的出邊。若還有點卻沒有入度 0 的點，就有環。以字母最小者優先，G′ 的過程如下。

取出	刪邊後可選集合
A	B,D
B	D,E
D	C,E
C	E
E	F
F	∅

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

輸出 A B D C E F，與 DFS 不同但同樣合法。每次取出者沒有尚未處理的前驅，這是不變量。若剩餘每點入度 ≥1，反覆沿前驅走，有限性保證重複頂點，得到環。FIFO 佇列可達 Θ(V+E)，字母最小堆則需 O(E+V log V)。

成本與自學檢核

鄰接表完整 DFS 的時間 Θ(V+E)，每頂點呼叫一次，每有向邊掃一次。無向邊掃兩次仍是 Θ(V+E)。鄰接矩陣每頂點掃 V 格，時間 $\Theta(V^2)$。三色、parent、d/f 及最深遞迴鏈共 Θ(V) 額外空間。

用明確堆疊模擬時，frame 應存 (u, 下一個鄰接索引)，掃完後才能寫 f[u]。只把所有鄰居 push 進頂點堆疊，未必重現本例的遞迴樹、完成時間與分類。

練習 A07-D01 至 D06 依序檢驗時間、分類、定理前提、無向模型、環證明與拓樸序。

來源 • MIT2004 pp.2–6; MIT2005 pp.2–5; CLRS3-22。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

連通問題的輸出不同

無向圖中，連通分量是可互相到達的極大頂點集合。有向圖中，SCC 要求兩方向皆有路徑；單向可達不足以合併。

問題	輸出	刪除測試
連通／強連通	頂點分割	無
割點	頂點集合	刪頂點及其相接邊後，分量數增加
橋	邊集合	刪邊後，分量數增加
點雙連通區塊	邊分組，割點可重複出現	區塊內無割點

本節無向輸入固定為有限、無自環、無平行邊的 simple graph。鄰接串列與根皆按字母序，時間從 1 起。來源 [S1, S2]。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

無向連通分量只需一個外層迴圈

對每個未標記頂點 s 開始 DFS 或 BFS，這次走訪全部標成同一個 component_id。 若存在從 s 到未標記 v 的路徑，路徑上第一個未標記頂點必由已走訪前驅發現，矛盾。因此走訪包含整個分量；走訪又只沿邊移動，不會跨越分量。 孤立點仍須由外層迴圈發現，不能只迭代邊。鄰接串列時間 Θ(V+E)，額外空間 O(V)。MIT 兩年份的 BFS 範例可支持這個遍歷背景，未提供 SCC 或割點演算法 [S3, S4]。

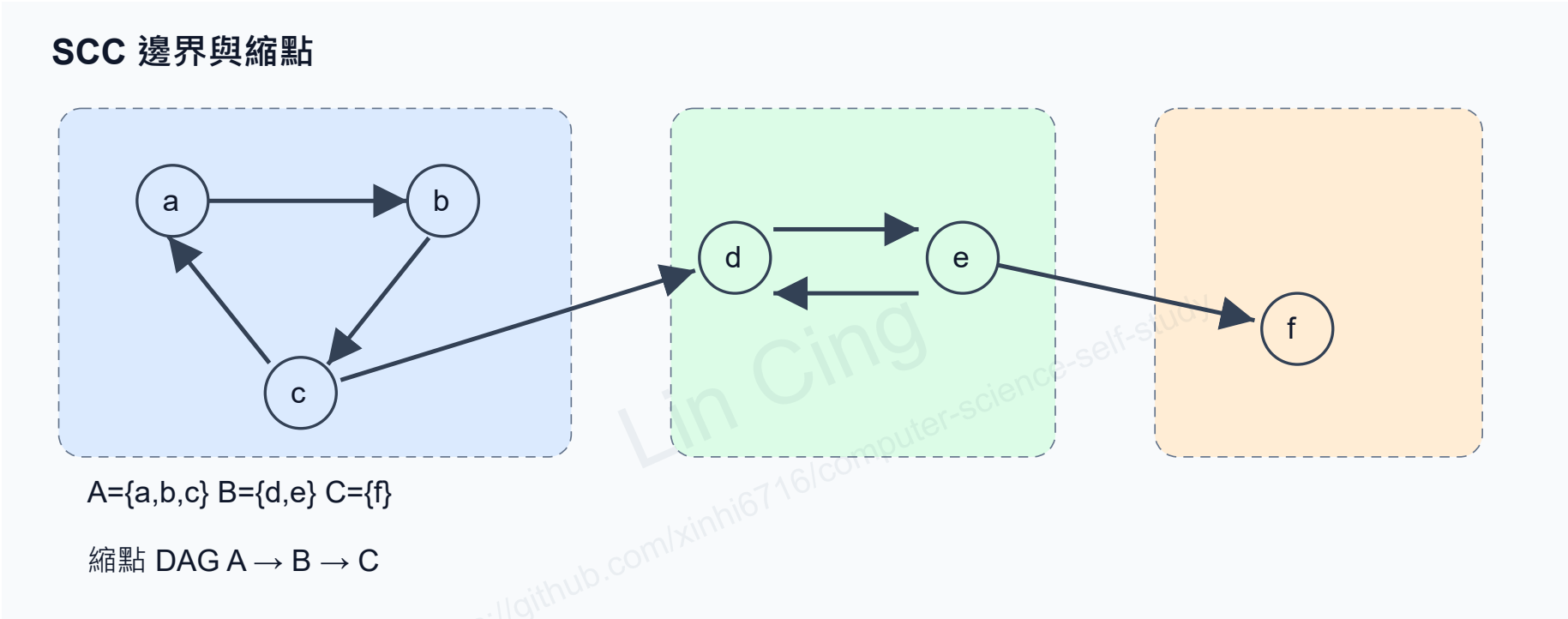
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

SCC 的極大性與縮點圖

u~v 定義為 u 可達 v 且 v 可達 u。長度零路徑給自反性，定義給對稱性，路徑串接給遞移性，所以 SCC 是等價類且分割 V。 每個 SCC 縮成一點，保留跨分量有向邊，刪去自環並可去重，得到 condensation graph。 若它有長度至少二的有向環，環上各分量都可互相到達，會合成一個更大的 SCC，違反極大性。因此縮點圖是 DAG。單點 SCC 不必有自環 [S1 §22.5]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

數例先固定邊與分量邊界



G 的邊為 ab, bc, ca, cd, de, ed, ef。此處 xy 表示 x→y。 SCC 為 A={a,b,c}、B={d,e}、C={f}；縮點為 A→B→C。DFS 在原圖從 a 一次可走遍六點，故「一棵原圖 DFS 樹就是一個 SCC」是錯的。

Kosaraju 第一趟要記完成時間

第一趟在 G 做完整 DFS 森林，每點完成時附加到 finish list。完成順序為 f,e,d,c,b,a；第二趟根候選順序取反向 a,b,c,d,e,f。

點	發現 d	完成 f	父點
a	1	12	∅
b	2	11	a
c	3	10	b
d	4	9	c
e	5	8	d
f	6	7	e

時間戳在發現與完成時各加一；不要把此 f 與頂點 f 混淆 [S1]。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

轉置保留 SCC，反轉跨分量邊

G^T 把每條 u→v 改為 v→u。原圖路徑 u⇝v 對應轉置路徑 v⇝u；故互相可達的等價類不變。 轉置串列為 a:[c], b:[a], c:[b], d:[c,e], e:[d], f:[e]。縮點為 C→B→A。 鄰接串列轉置先建 V 個空串列，再掃每條邊加入反向串列，成本 Θ(V+E)。第二趟必清空 visited，保留第一趟的完成順序 [S1]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完成時間引理分兩種先發現情況

設不同 SCC 間有 $C \to D$，並定義 $F(C)=\max\{f(v):v\in C\}$、$d(C)=\min\{d(v):v\in C\}$。要證 $F(C)>F(D)$。

1. 若 $d(C)<d(D)$，C 最先被發現的 x 當時可沿全白路徑到 C 的每點，也可經跨邊到 D 每點。這些點皆在 x 返回前完成，所以 $F(C)=f(x)>F(D)$。
2. 若 $d(D)<d(C)$，D 最先被發現的 y 可走遍 D，但不可到達 C，否則 $C \to D$ 與 $D \rightsquigarrow C$ 會合併兩個 SCC。故 D 的所有點在 C 任一點被發現前完成，$F(D)<d(C)<F(C)$。

此引理比較的是分量最大完成時間，不保證每個 C 中頂點都比每個 D 中頂點晚完成 [S1 Lemma 22.14]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第二趟恰好收一個 SCC 的證明

依第一趟 f 遞減選尚未訪問的根 u，設它在 C。歸納假設先前樹已各收完整 SCC，故 C 此時全白，而且 F(C) 在剩餘分量中最大。 在 G^T 若有 C→D，原圖就有 D→C，完成引理給 F(D)>F(C)。因此 D 必已在第二趟訪問。從 C 不能走進其他未訪問分量。 另一方面，C 在 G^T 仍強連通，且全白，故 DFS 從 u 必走遍 C。這棵樹既不漏 C，也不包含別的分量。空森林為歸納基礎，逐樹成立直到全圖完成。 兩次 DFS 加轉置總時間 Θ(V+E)，額外空間 O(V+E)，含轉置 [S1 Theorem 22.16]。

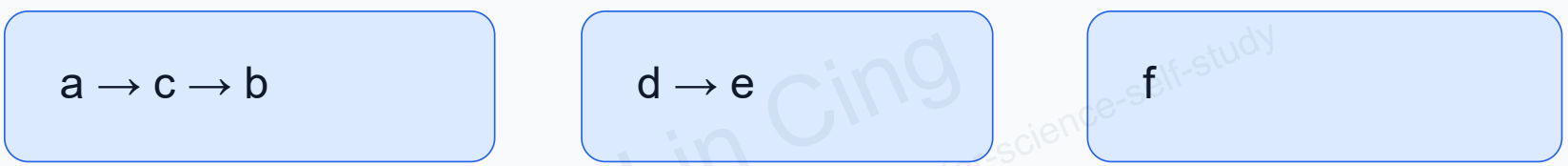
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第二趟的根、堆疊與跳過

Kosaraju 兩趟方向與順序

第一趟 G 的完成順序 f,e,d,c,b,a

反轉作根順序 a,b,c,d,e,f



第二趟在 G^T，每框是一棵樹 / 一個 SCC

已訪問的 b,c,e 根候選皆跳過

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第二趟的根、堆疊與跳過 (續)

根候選	G^T 遞迴堆疊最深狀態	新發現順序	樹輸出
a	[a,c,b]	a,c,b	{a,b,c}
b,c	已訪問，跳過	∅	∅
d	[d,e]；邊 d→c 已黑	d,e	{d,e}
e	已訪問，跳過	∅	∅
f	[f]；邊 f→e 已黑	f	{f}

每棵樹返回後呼叫堆疊清空；分量輸出不依第二趟完成時間重排。使用 G 而不轉置、但維持 f 遞減時，根 a 將錯誤收進所有六點。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Tarjan 使用尚未定案的頂點堆疊

disc[u] 是第一次發現序號；low[u] 是此版遞迴更新的 lowlink。S 儲存已發現但尚未輸出 SCC 的點；onstack[u] 表示 u∈S，與 DFS 灰色不同。

```
visit(u):
  disc[u] = low[u] = ++clock; push(S,u); onstack[u]=true
  for v in Adj[u]:
    if disc[v] is unset:
      visit(v); low[u]=min(low[u],low[v])
    else if onstack[v]: low[u]=min(low[u],disc[v])
  if low[u]==disc[u]:
    repeat: x=pop(S); onstack[x]=false; emit x
    until x==u
```

外層依序 visit 所有未發現點。已輸出點不再降低 low；對已發現且仍在 S 的邊使用 disc[v]，不可任換 low[v]。來源 [S5] 的現代記號改寫。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Tarjan 的完整返回與彈出紀錄

Tarjan 分量堆疊與彈出邊界

堆疊 S 由底到頂；最右為 top

[a,b,c,d,e,f] → pop f

[a,b,c,d,e] → pop e,d

[a,b,c] → pop c,b,a

low = [1,1,1,4,4,6] SCC = {f} | {d,e} | {a,b,c}

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

下表 S 由底到頂。disc 按 a,b,c,d,e,f 為 1,2,3,4,5,6。

Tarjan 發現頂點與更新 low

事件	當下更新	s
進 a	disc[a]=low[a]=1	[a]
進 b	disc[b]=low[b]=2	[a,b]
進 c	disc[c]=low[c]=3	[a,b,c]
c→a	low[c]=1	[a,b,c]
進 d	disc[d]=low[d]=4	[a,b,c,d]
進 e	disc[e]=low[e]=5	[a,b,c,d,e]
e→d	low[e]=4	[a,b,c,d,e]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Tarjan 返回與輸出分量

事件	當下更新	S
進 f	disc[f]=low[f]=6	[a,b,c,d,e,f]
f 返回，6=6	彈 f；輸出 {f}	[a,b,c,d,e]
e 收 f，返回	low[e]=min(4,6)=4；不彈	[a,b,c,d,e]
d 收 e，4=4	low[d]=4；彈 e,d；輸出 {d,e}	[a,b,c]
c 收 d，返回	low[c]=min(1,4)=1	[a,b,c]
b 收 c，返回	low[b]=1	[a,b,c]
a 收 b，1=1	low[a]=1；彈 c,b,a；輸出 {a,b,c}	[]

最後 low 為 1,1,1,4,4,6；各點 onstack 最終皆 false。b 返回時已非灰色，但尚在 S，所以呼叫堆疊與 S 不相同。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

SCC lowlink 與無向 low 的分界

項目	有向 Tarjan SCC	無向割點／橋
非樹邊目標	必須仍 onstack	必須是祖先，排除父邊反向
狀態堆疊	未定案頂點	雙連通用邊堆疊
邊界測試	low[u]=disc[u]	low[v]≥disc[u] 或 >
輸出	頂點分割	割點、橋、邊區塊

SCC 邊可指向已完成呼叫但仍在 S 的點。例 a→b,b→a,a→c,c→b，b 呼叫返回後 c→b 仍影響 low[c]。若只用 gray 判定會誤分 c。 若從 x 先輸出單點 SCC，再從 y 看見 y→x，onstack[x]=false，這條邊不能把 y 與 x 合併。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

無向 low 是子樹的逃生能力

對 DFS 頂點 u，low[u] 是從 u 子樹沿零或多條向下樹邊，再至多一條回祖先邊可達的最小 disc；包含 disc[u]。初始化 low[u]=disc[u]。樹子點 v 返回後取 min(low[u],low[v])。非父邊連到已發現祖先 w 時取 min(low[u],disc[w])。 無向 DFS 無跨子樹邊，否則先走的子樹會經那條邊發現另一子樹。故判斷子樹能否繞過 u，只需知道能否由回邊到達 u 的真祖先 [S1 Problem 22-2]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

割點判定必須分根與非根

非根 u 是割點，當且僅當存在 DFS 子點 v 滿足 low[v]≥disc[u]。此時 v 子樹無法到達 u 的真祖先，刪 u 便與 u 的父側分離；若每個子樹都可回真祖先，刪 u 後仍可經祖先側連接。 根 r 是割點，當且僅當 DFS 樹子點數至少 2。不同根子樹無邊相連，刪根會拆開；只有一棵子樹時，刪根仍由那棵子樹連通。零個子點也不是割點。 根不能套非根公式。只有 a—b 的圖也有 low[b]≥disc[a]，但刪 a 不增加分量數 [S1 22-2(a,b)]。

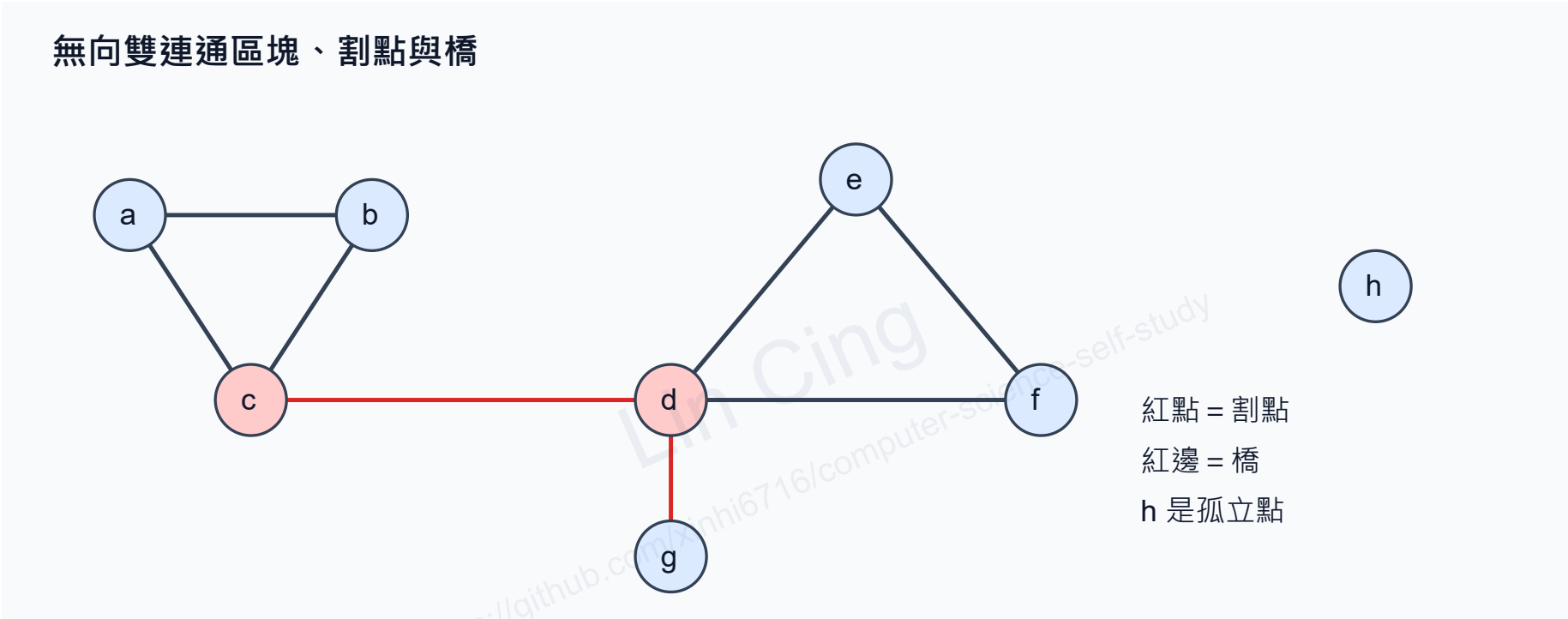
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

橋使用嚴格大於

樹邊 (u,v) 是橋，當且僅當 low[v]>disc[u]。沒有回邊到 u 或更早祖先，刪除該邊後兩側分離。 等號 low[v]=disc[u] 表示子樹能回到 u，形成包含 (u,v) 的環，故該邊不是橋。但刪掉 u 同時刪回邊終點，非根 u 仍可能是割點。 在無向 simple graph，非樹邊總在環中，因此不會是橋。若擴充到多重圖，必以 edge_id 跳過恰好那條 parent edge，不能跳過所有通往父點的邊；兩條平行邊就不是橋。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

無向數例與完整 disc／low 表



邊集合 ab,bc,ca,cd,de,ef,fd,dg；h 孤立。字母序 DFS。

無向 DFS 的 disc／low：a 至 d

點	disc	low	父點	DFS 子點
a	1	1	∅	b
b	2	1	a	c
c	3	1	b	d
d	4	4	c	e,g

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

無向 DFS 的 disc／low：e 至 h 與判定

點	disc	low	父點	DFS 子點
e	5	4	d	f
f	6	4	e	∅
g	7	7	d	∅
h	8	8	∅	∅

回邊 c→a 降 low[c] 至 1，f→d 降 low[f] 至 4，再逐層向上傳遞。割點 {c,d}，橋 {cd,dg}。a 只有一個樹子點，不是割點；d 有多個子點但它是非根，仍套非根規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙連通區塊需要邊堆疊

採 block 慣例，橋獨立成二點一邊區塊，孤立點另記 singleton vertex block。大小至少三的點雙連通區塊為極大無割點子圖。這與 CLRS 22-2 的只分割非橋邊的 cycle-based bcc 慣例不同，輸出時須說明。 走樹邊 (u,v) 前 push 該邊；看到往祖先的回邊時 push 一次。子點返回且 low[v]≥disc[u] 時，持續 pop 並把 (u,v) 本身也包含進輸出的區塊，然後停止。 這個邊界適用根的每個子樹，與根是不是割點分開判定。每邊最多 push/pop 一次，總時間 Θ(V+E)。區塊分割邊，割點可以同時屬於多個區塊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

邊堆疊的每個分量邊界

下列堆疊由底到頂；ca 與 fd 是回邊，父邊反向不再 push。

事件	邊堆疊	輸出
依序 push ab,bc,ca,cd,de,ef,fd	[ab,bc,ca,cd,de,ef,fd]	∅
f 返回 e，4<5	不變	∅
e 返回 d，4≥4	[ab,bc,ca,cd]	pop fd,ef,de
push dg，g 返回，7≥4	[ab,bc,ca,cd]	pop dg

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

邊堆疊回到根並輸出區塊

事件	邊堆疊	輸出
d 返回 c，4≥3	[ab,bc,ca]	pop cd
c 返回 b，1<2	不變	∅
b 返回 a，1≥1	[]	pop ca,bc,ab
以 h 為新根	[]	孤立點 {h}

區塊為 {de,ef,fd}、{dg}、{cd}、{ab,bc,ca}，加孤立點 {h}。只 pop 一條邊、或在遇到 de 之前停止，會把三角形拆錯。空堆疊是每棵完整 DFS 樹結束的檢查條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把公式用在故障問題上

本例原有兩個連通分量，主圖與 {h}。刪 c 後為 {a,b}、{d,e,f,g}、{h} 共三個；刪 d 後為 {a,b,c}、{e,f}、{g}、{h} 共四個。刪橋 cd 後為三個，刪 de 仍為兩個。 刪掉割點所增加的分量數不必是 1。橋表示單一連線故障，割點表示節點及所有相接線路一起故障；SCC 則描述方向約束下的互相可達，三者不能互換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

來源與適用範圍

S1 為 CLRS 第三版 §22.5，印刷頁 615–620；割點與橋補充用 Problem 22-2，印刷頁 621–622。S2 為原手寫 note P34–36 的 DFS、完成時間與邊分類，這些已讀頁無 SCC 或割點 low 公式。 S3 為 MIT 2004 L14，PDF 45–59；S4 為 MIT 2005 L17，同範圍為 BFS，僅補遍歷背景。S5 為 Tarjan 1972 原始論文，SCC 部分以現代 disc/low/onstack 記號改寫。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

CLRS, Introduction to Algorithms, 3rd ed., §§22.1–22.5 and Problem 22-2. Bridges are included here as one-edge blocks. Original handwritten notes 演算法_wjungle, PDF pp. 34–36, 40, for DFS notation and BFS augmenting paths. MIT 6.046J Fall 2004, L13 pp. 2–6 and L14 pp. 43–59; Fall 2005, lec16 pp. 2–5 and lec17 pp. 43–59. Representation and BFS background. MIT 6.006 Fall 2011 Lecture 14 pp. 2–6; Spring 2020 Lecture 10 pp. 2–4. DFS and topological order. Robert Tarjan, “Depth-First Search and Linear Graph Algorithms,” SIAM J. Comput. 1(2), 146–160 (1972), doi:10.1137/0201010. SCCs.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study